

FGS-SLAM: A Calibration-Free SLAM Framework Using Feed-Forward 3D Gaussian Splatting

Xuefeng Yang^{1*}Tongtai Cao¹Yue Liu^{1,2†}

¹ Beijing Engineering Research Center of Mixed Reality and Advanced Display
School of Optics and Photonics, Beijing Institute of Technology

² Zhengzhou Research Institute, Beijing Institute of Technology

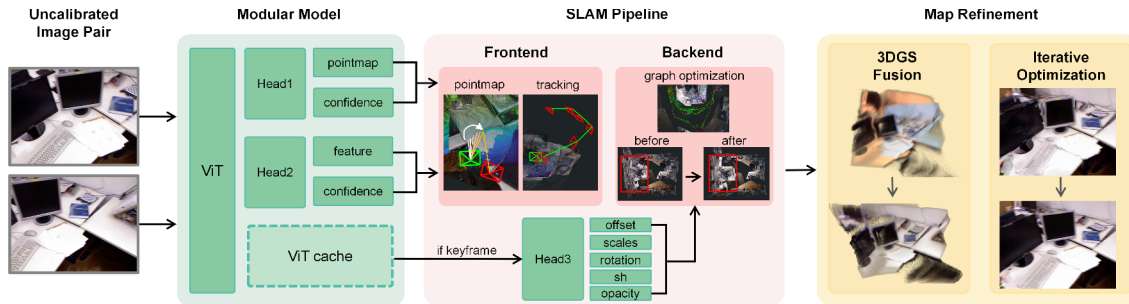


Figure 1: Overview to the proposed framework. Given a pair of images, We first perform feed-forward 3D reconstruction using a Vision Transformer-based model, and then feed the results into a SLAM system for tracking and mapping. An effective map refinement process further processes the SLAM outputs.

ABSTRACT

3D Gaussian Splatting (3DGS)-based Simultaneous Localization and Mapping (SLAM) has recently emerged as a promising paradigm for AR/VR, enabling real-time tracking and high-fidelity photorealistic rendering. However, existing methods typically require accurate camera intrinsic calibration and suffer from slow 3DGS map construction, limiting immersive experiences. We present a plug-and-play monocular 3DGS-SLAM system that employs a modular feed-forward model to infer 3D Gaussian parameters from two uncalibrated views, together with a map refinement strategy tailored for feed-forward Gaussians. This is the first monocular SLAM framework leveraging a two-view 3DGS reconstruction prior, achieving real-time performance at 20 FPS.

Index Terms: 3D Gaussian Splatting, SLAM, 3D Reconstruction.

1 INTRODUCTION

Visual SLAM serves as a fundamental building block of modern AR/VR systems and robotic platforms, enabling accurate camera tracking and real-time mapping. However, traditional pointcloud-based pipelines struggle to reconstruct high-quality, texture-rich scenes or support novel view synthesis, limiting the visual fidelity required for immersive AR/VR experiences.

Recent advances in 3D Gaussian Splatting enable photorealistic scene reconstruction from posed images [1]. While several systems [2] incorporate 3D Gaussian primitives into SLAM, they remain limited by costly iterative optimization and lack of robustness in real-world application.

We propose a real-time SLAM framework based on modular two-view 3DGS priors [5], unifying tracking and mapping. Efficient 2D-3D matching, robust map maintenance, and scalable

optimization enable accurate frame-to-frame pose estimation with global consistency. A dedicated refinement module improves multi-view Gaussian reconstruction. The proposed system is plug-and-play, requiring neither camera calibration nor depth sensors, and is well suited for consumer-level AR/VR applications.

2 METHOD

Our framework comprises three components: a feed-forward reconstruction module, a SLAM pipeline, and an offline optimization.

2.1 Feed-forward Model for 3DGS Reconstruction

3DGS represents a scene with a set of Gaussian ellipsoids. Each Gaussian is parameterized by its mean position μ , opacity α , rotation q , scale s , and color c . Given these parameters, the color image is obtained via splatting and blending [1].

Given an image pair, our model encodes the input with a Vision Transformer (ViT) and applies DPT-style heads [4] on the decoder features for dense 3D reconstruction. Following MAST3R, two geometry heads predict point maps, feature maps, and the corresponding confidence scores, while a Gaussian head following Splatt3R [5] estimates pixel-align 3DGS parameters, with Gaussian means defined as offsets from the pointmaps.

To mitigate the inference overhead introduced by the Gaussian head, we adopt a modular design that decouples it from the main SLAM pipeline. During frontend tracking, only the geometry heads are executed for fast pose estimation. ViT features of selected keyframes are cached and shared with the backend, where the Gaussian head is applied exclusively to new keyframes. This decoupled design preserves reconstruction quality while improving system efficiency, yielding a 2 FPS speedup in our experiments.

2.2 SLAM Pipeline

Our SLAM system mainly consists of three components: matching, frontend tracking, and backend optimization and mapping.

Matching. Assuming a generic central camera model and accounting for scale ambiguity, we independently project each point

*e-mail: yxf_bit@163.com

†e-mail: liuyue@bit.edu.cn

$x \in X_i^j$ by iteratively optimizing its pixel coordinate p in the reference frame, minimizing the ray-based reprojection error[3]:

$$\mathbf{p}^* = \underset{\mathbf{p}}{\operatorname{argmin}} \left\| \psi([\mathbf{X}_i^j]_{\mathbf{p}}) - \psi(\mathbf{x}) \right\|^2. \quad (1)$$

The pixel updates are solved iteratively via Levenberg-Marquardt, followed by coarse-to-fine feature matching using $\mathbf{C}_i^l$, $\mathbf{C}_i^j$, both implemented in CUDA kernels provided by [3].

Frontend. Given the last keyframe pointmap estimate $\tilde{\mathbf{X}}_{k,j}^k$ and current pointmap accessed via $F(\mathbf{I}_f, \mathbf{I}_k)$, we estimate the current camera pose by minimizing a ray-based ICP objective [3]:

$$E_r = \sum_{i,j \in m_{f,k}} \left\| \frac{\psi(\tilde{\mathbf{X}}_{k,j}^k) - \psi(\mathbf{T}_{kf} \mathbf{X}_{f,i}^f)}{\omega} \right\|_{\rho}. \quad (2)$$

where ω denotes a per-match weighting following [3], and $\|\cdot\|_{\rho}$ represents the Huber norm. Meanwhile, the point map $\tilde{\mathbf{X}}_k^k$ is continuously refined using a weighted filter over incoming frames.

Backend. Upon keyframe insertion, we perform online factor-graph optimization with temporal and loop-closure constraints. Loop closures are detected via coarse-to-fine ASMK using ViT features, followed by Gauss–Newton optimization [3].

2.3 Map Refinement

To maintain map consistency after online reconstruction, we perform a map refinement stage with two components.

Gaussian fusion. While dense model outputs benefit robust tracking, they introduce redundancy when multiple keyframes observe the same scene region, leading to duplicated and misaligned 3D Gaussian ellipsoids. As a result, such redundancy causes blurring and ghosting artifacts and incurs unnecessary memory overhead, as illustrated in Fig. 1 (3DGS Fusion).

To address this issue, we fuse corresponding Gaussians from adjacent keyframes using cached frontend matching information. Each matched Gaussian pair is merged into a unified representation using filtering, with a pixel-wise mask to avoid redundant computation. This fusion reduces the number of Gaussians by about 20% while improving spatial consistency.

Iterative optimization. To further mitigate overfitting and stitching artifacts introduced by feed-forward Gaussian prediction, we perform iterative optimization on the fused map. We optimize the Gaussians using photometric supervision, while the objective combines an L1 loss and SSIM. This iterative refinement effectively improves visual coherence across views and further enhances rendering quality.

3 RESULTS

We conduct our experiments on an NVIDIA RTX 4090D GPU, and run evaluation on the TUM dataset and self-captured sequences in calibrated mode. For tracking performance, we report the RMSE of ATE in meters. We demonstrate state-of-the-art tracking performance compared with the DROID-SLAM[6] as shown in Tab.1. Notably, in the 360 scene, our uncalibrated mode even achieves a lower ATE than the calibrated DROID-SLAM.

Furthermore, Fig. 2 presents qualitative and quantitative comparisons with MonoGS on novel view rendering across diverse scenes, including the TUM dataset and our self-captured RealSense sequences. In outdoor scenarios, our method achieves more accurate tracking and mapping results. These results highlight the robustness of our method in reconstructing unknown scenes, demonstrating its strong generalization capability in real-world scenarios.

Table 1: Tracking Performance on TUM datasets compared with DROID-SLAM [6] (ATE RMSE ↓ [m]).

Setting	Method	360	desk2	room
Calibrated	DROID-SLAM	0.111	0.042	0.049
	Ours	0.052	0.024	0.051
Uncalibrated	DROID-SLAM	0.202	0.091	0.918
	Ours	0.070	0.055	0.118

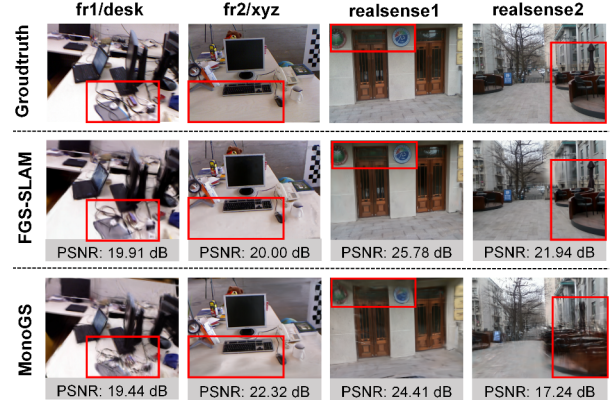


Figure 2: Results of novel view rendering compared with MonoGS [2] on the TUM dataset and our self-captured RealSense sequences.

4 CONCLUSION

We present a robust 3DGS-based SLAM system that enables accurate pose estimation and feed-forward photorealistic reconstruction. A modular deployment strategy reduces redundant computation and significantly improves efficiency, achieving strong performance in both tracking and reconstruction. Our map refinement balances efficiency and reconstruction quality for two-view scene reconstruction, making the framework well suited for immersive AR/VR applications.

ACKNOWLEDGMENTS

This work was supported by the National Key R&D Program of China (No.2024YFB2808804)

REFERENCES

- [1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023. 1
- [2] H. Matsuki, R. Murai, P. H. Kelly, and A. J. Davison. Gaussian splatting slam. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 18039–18048, 2024. 1, 2
- [3] R. Murai, E. Dexheimer, and A. J. Davison. Mast3r-slam: Real-time dense slam with 3d reconstruction priors. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 16695–16705, 2025. 2
- [4] R. Ranftl, A. Bochkovskiy, and V. Koltun. Vision transformers for dense prediction. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 12179–12188, 2021. 1
- [5] B. Smart, C. Zheng, I. Laina, and V. A. Prisacariu. Splatt3r: Zero-shot gaussian splatting from uncalibrated image pairs. *arXiv preprint arXiv:2408.13912*, 2024. 1
- [6] Z. Teed and J. Deng. Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras. *Advances in neural information processing systems*, 34:16558–16569, 2021. 2